

Agentic Coding



gollnickdata.de

Agenda

Was Sie erwarten können



1. What is an Agent?
2. Coding Agents on the market
3. How do Coding Agents work?
4. Agentic Coding Introduction
5. Agentic Coding Levels
6. Socio-economic impact



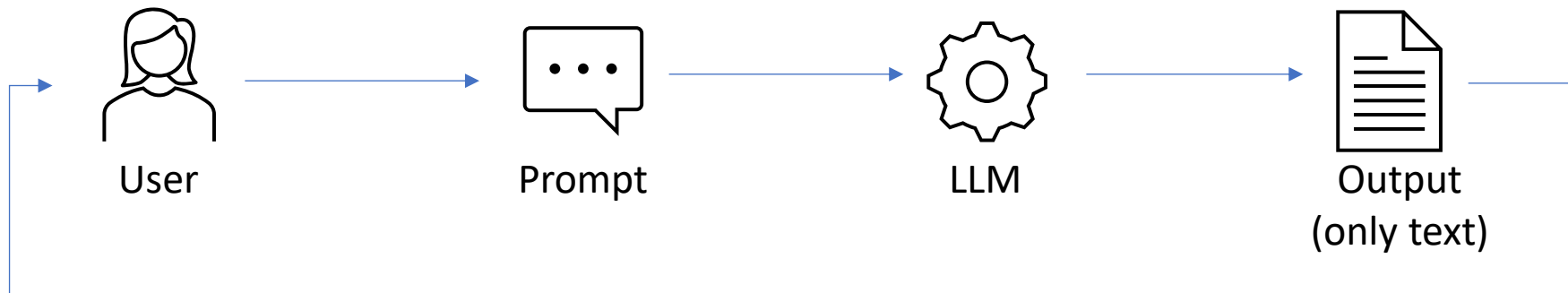
gollnickdata.de

What is an Agent?

Agents

What is an Agent?

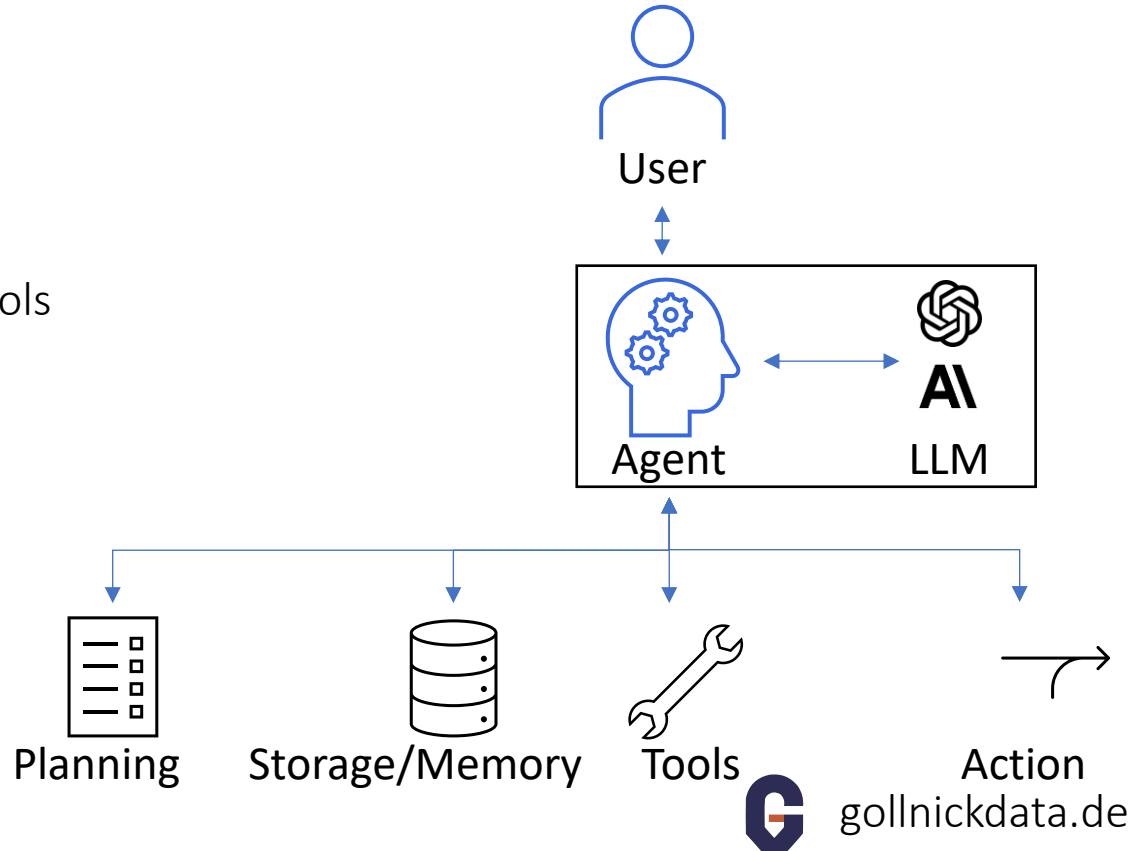
Chatbot



Agents

What is an Agent?

- LLM apps execute tasks
- core element: agent
- uses planning, memory, and tools
- can perform actions
- is an expert in its field



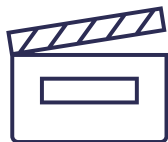
Agents

What is an Agent?

Perceives
Environment



Takes Actions
autonomously



To achieve a
defined goal



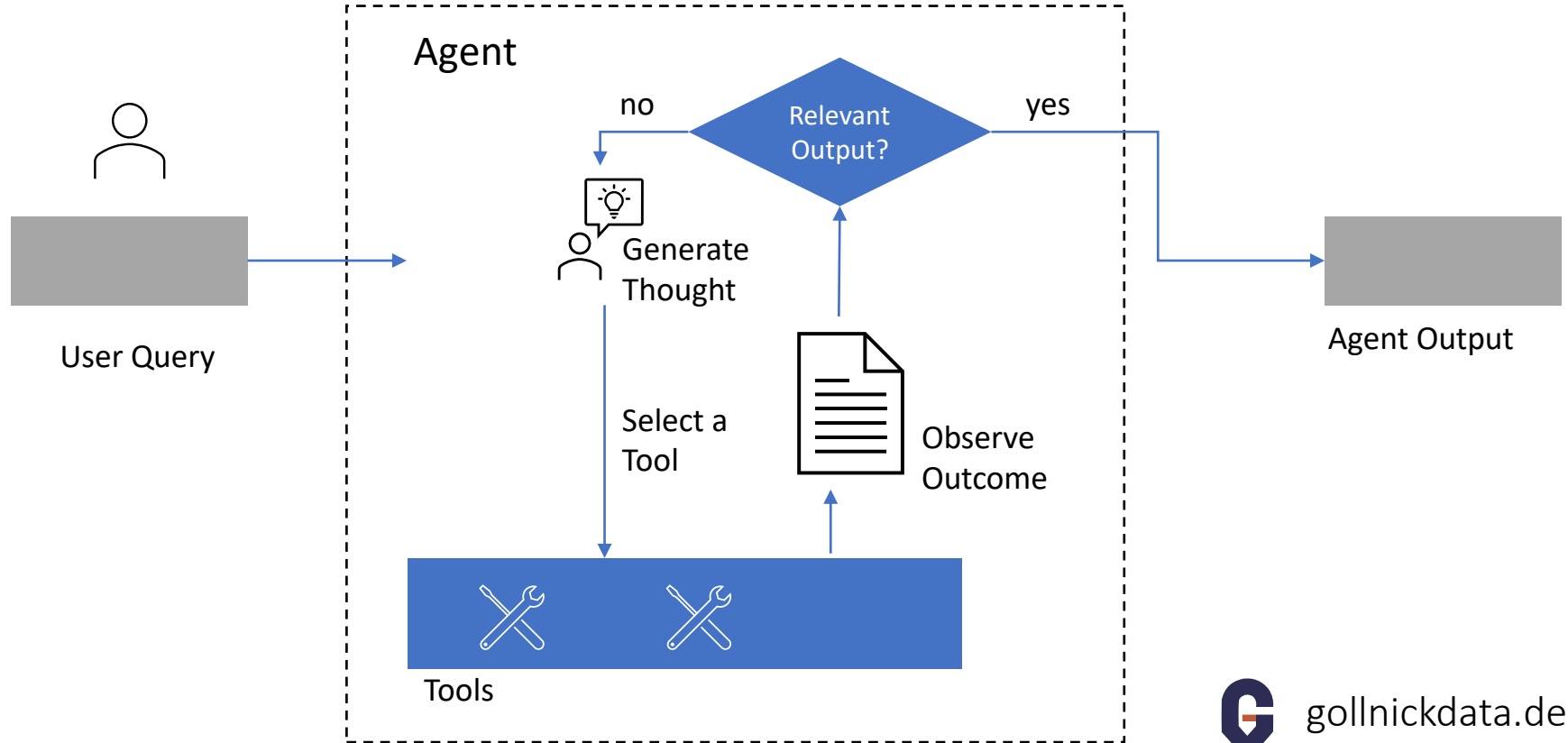
Improve
through
learning



gollnickdata.de

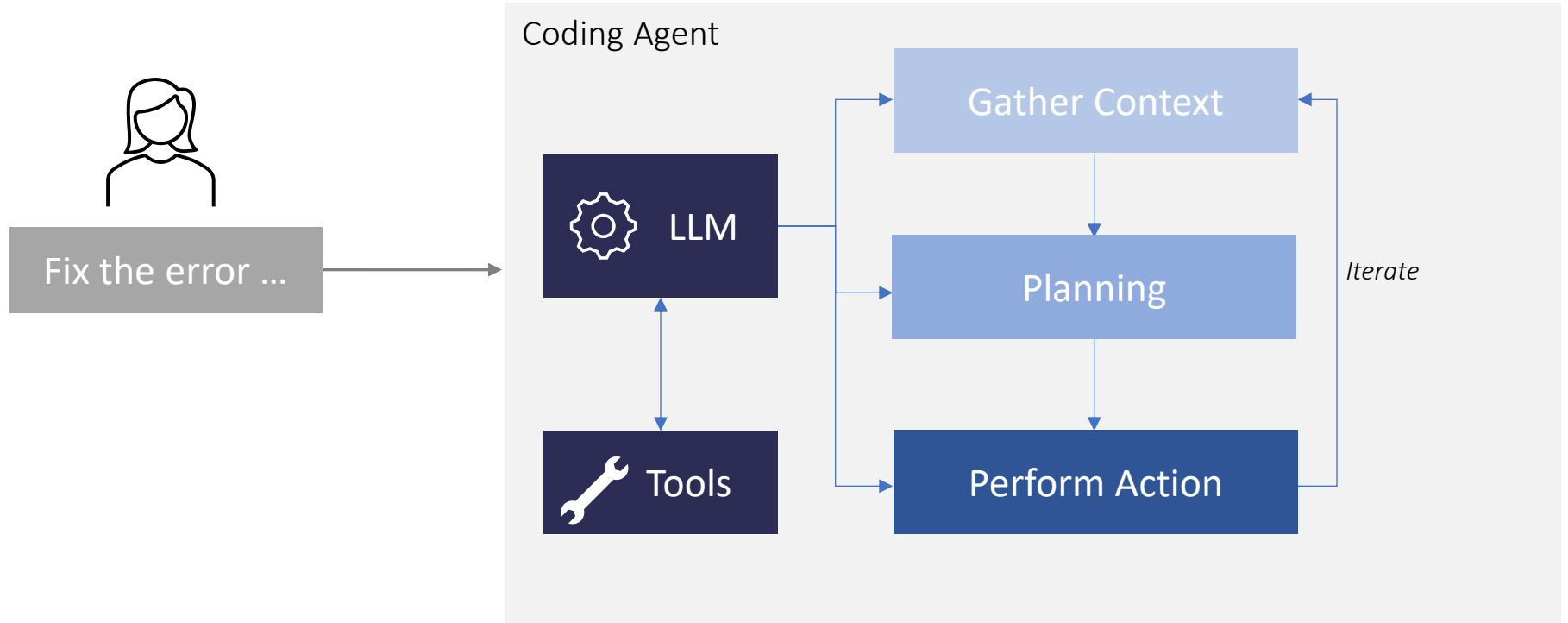
Agents

How does an Agent work?



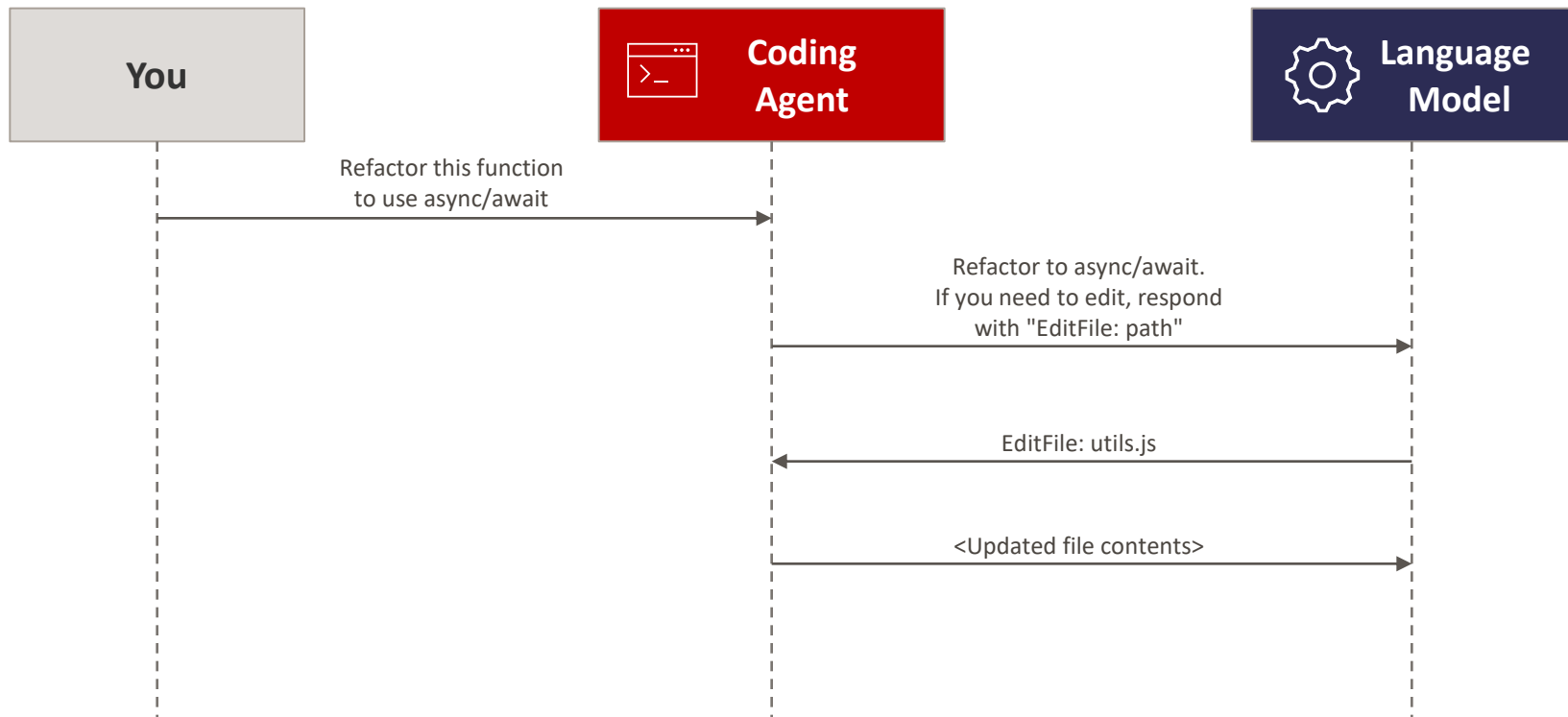
Agents

What is a Coding Agent?



Claude Code

How the agentic coding loop works



Coding Agents on the market

I don't think I've typed like a line of code probably since December, basically, which is an extremely large change.

Andrej Karpathy, AI Researcher

Agents

Coding Agents: Evolution

Auto Completion



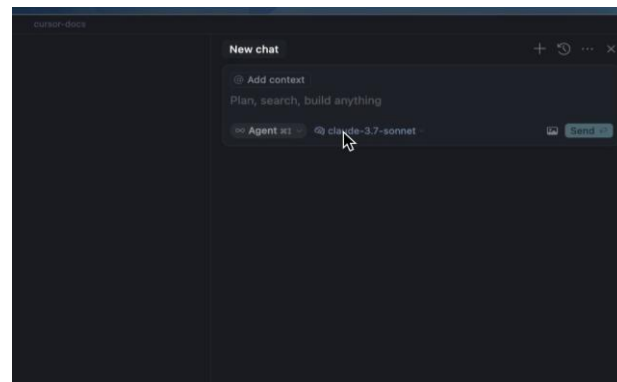
Inline Complete



Complete code base

```
geometry.at = vec![vx, vy, vz];  
}  
  
Edit selected code  
Auto  
Edit Selection  
  
Mesh::process_face(  
  vx,  
  vy,
```

```
23 You, 9 seconds ago - 1 Author (You) 0 implementations  
24 pub struct EncodedMessageQueue {  
25     pub queue: Vec<EncodedMessage, ClientFilter>,  
26  
27     sender: Arc<Sender<Vec<EncodedMessage, ClientFilter>>>,  
28     receiver: Arc<Receiver<Vec<EncodedMessage, ClientFilter>>>,  
29 }  
30  
31 [imp] EncodedMessageQueue { You, 8 seconds ago - Uncommitted changes  
32     pub fn new() -> Self {  
        let (sender, receiver) = crossbeam_channel::unbounded();  
        Self {  
            queue: vec![],  
            sender: Arc::new(sender),  
            receiver: Arc::new(receiver),  
        }  
    }  
}
```



- Proposes automatically next code tokens

- Edits multiple lines
- Available in VS Code, Cursor, Windsurf

- Accesses complete code base
- Performs edits



Agents

Vibe Coding vs. Agentic Coding

Applications of Vibe Coding



Rapid Prototyping



UI/UX Development



Learning & Exploration



Debugging

Applications of Agentic Coding



Enterprise Software



Infrastructure as Code



Data Pipelines



Maintenance

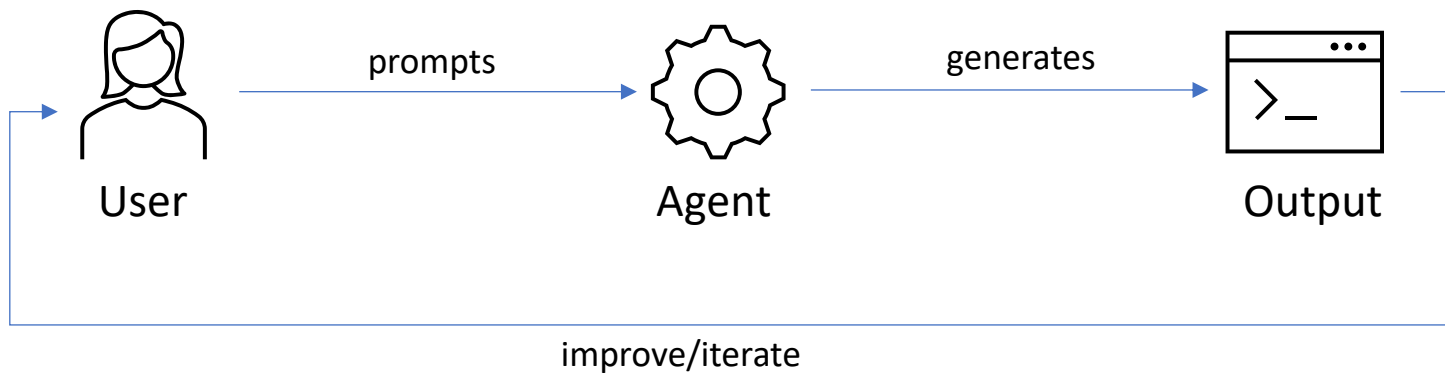


gollnickdata.de

Agents

Vibe Coding vs. Agentic Coding

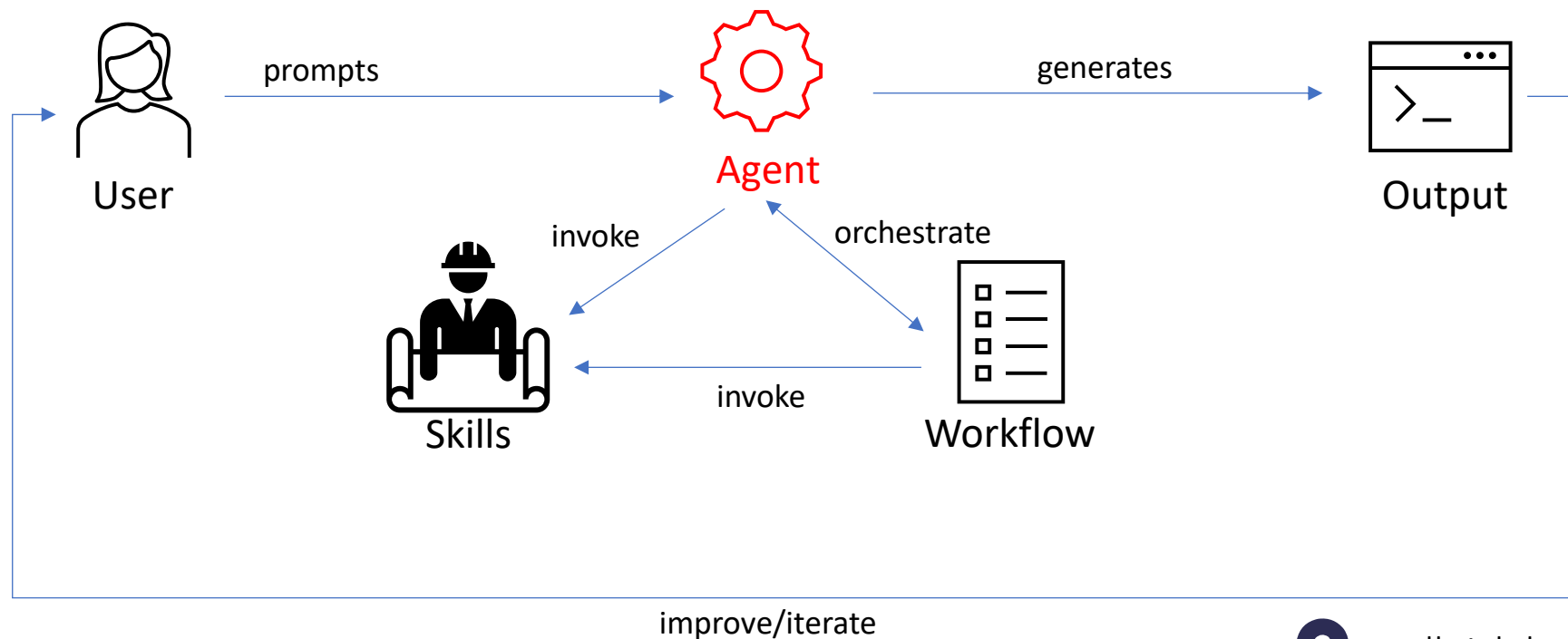
Vibe Coding



Agents

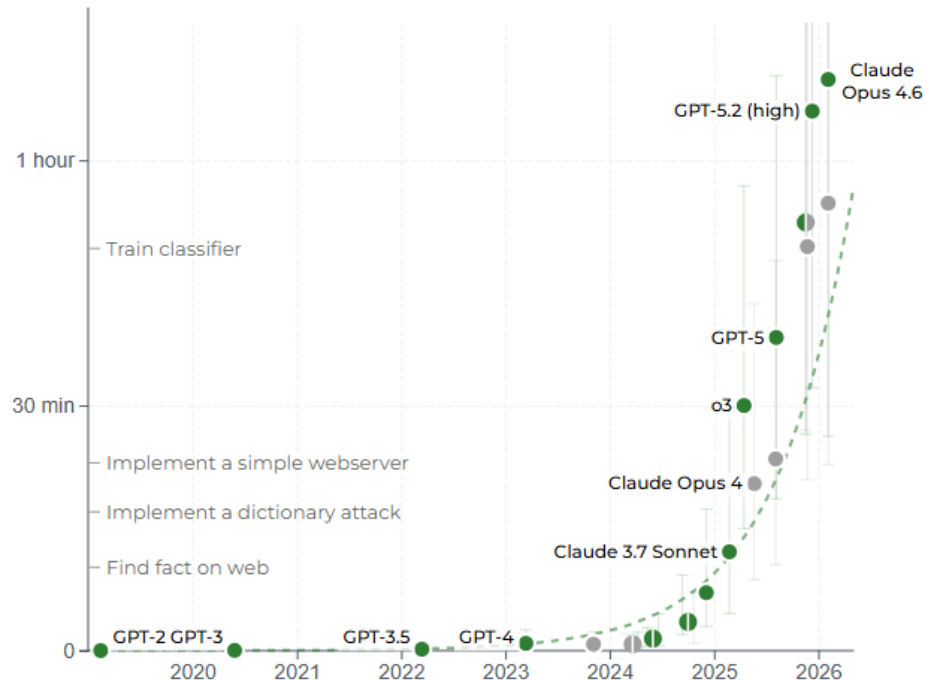
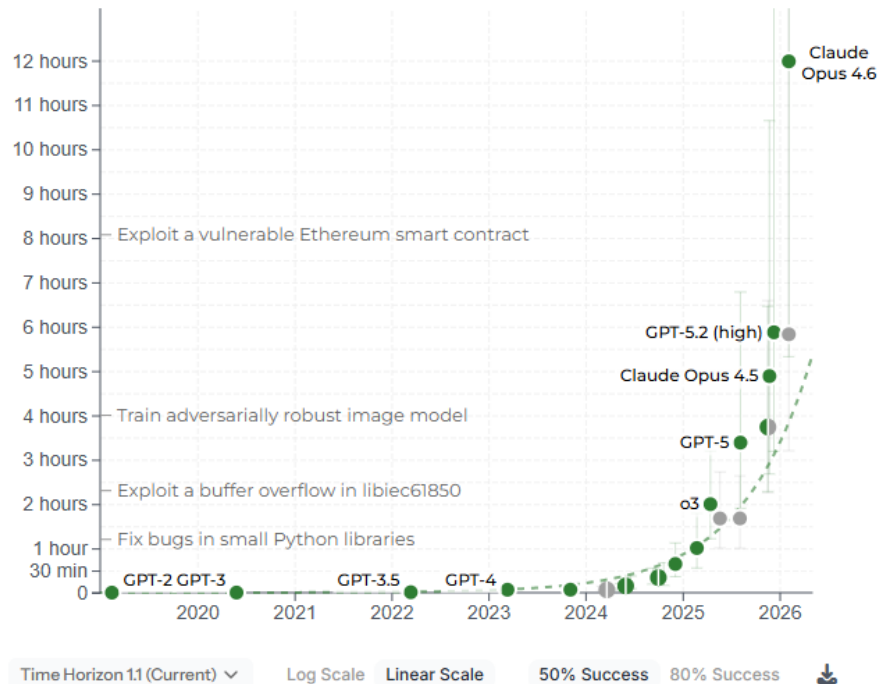
Vibe Coding vs. Agentic Coding

Agentic Coding



Agents

Coding Agents: Ability to perform long tasks



Source: <https://metr.org/>

Agents

(Vibe) Coding Agents: Browser-Based



- Browser-based
- freemium
- <https://replit.com/>



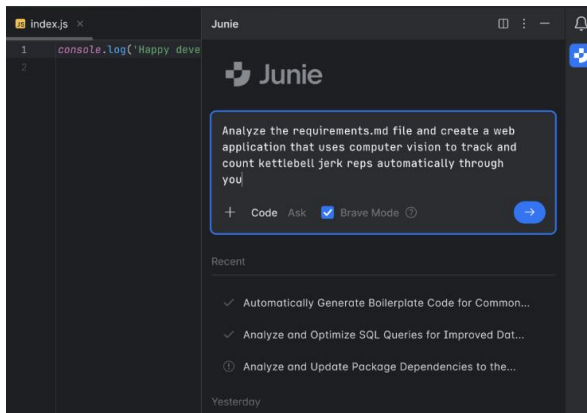
- Browser-based
- freemium
- <https://lovable.dev/>



Agents

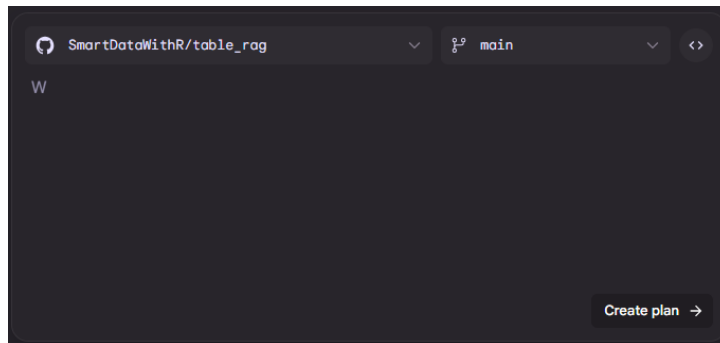
Coding Agents: Browser-Based

Junie



- Developed by JetBrains
- Available in JetBrains-IDEs (e.g. PyCharm)

Jules



- Browser-based
- Developed by Google
- Integrates with GitHub



Agents

Coding Agents: Terminal



- Works from command line
- Can work with your complete code base
- Install: `npm install -g @anthropic-ai/claude-code`
- Available for claude subscribers or API-use



- [Source](#)
- Can work with your complete code base
- Install: `npm install -g @google/gemini-cli`
- Available via API key or Google SignIn
- 100 Gemini 2.5 Pro requests free per day!



Agents

Coding Agents: Terminal



- [Source](#)
- Open Source
- Can work with your complete code base
- Less secure, but lets you look into Claude Code

Install: `git clone https://github.com/ultraworkers/claw-code`

`cd claw-code/rust`
`cargo build --workspace`

- Available for any LLM Provider



- [Source](#)
- Can work with your complete code base
- Install: `curl -fsSL https://opencode.ai/install | bash`
- Available via API key



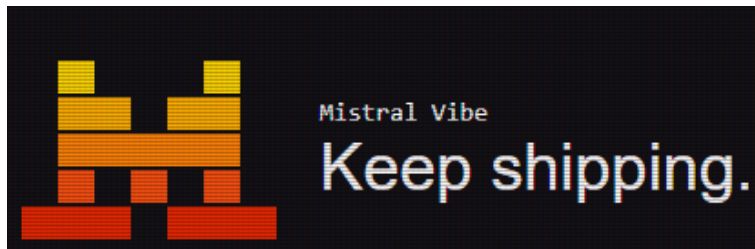
gollnickdata.de

Agents

Coding Agents: Terminal

Codex

- Developed by OpenAI
- Included in ChatGPT Plus, Pro, Business, Edu, Enterprise, or via API-key
- Works from command line



- Developed by Mistral
- Released under Apache 2 license



gollnickdata.de

Agents

Coding Agents: IDE



Cursor

- 20\$/month
- Suitable for
 - Experienced devs, large codebase
- Makes Claude 4 available



Windsurf

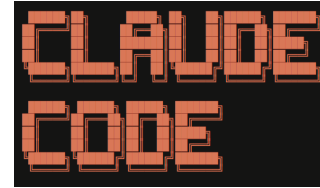
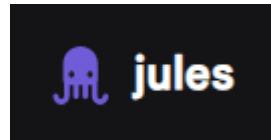
- 15\$/month
- Suitable for
 - Beginner, vibe-coding, personal projects
- Develop web apps quick and without configuration effort



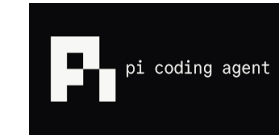
gollnickdata.de

Agents

Coding Agents: IDE



Hermes Agent



User Experience Level

Beginner

Intermediate

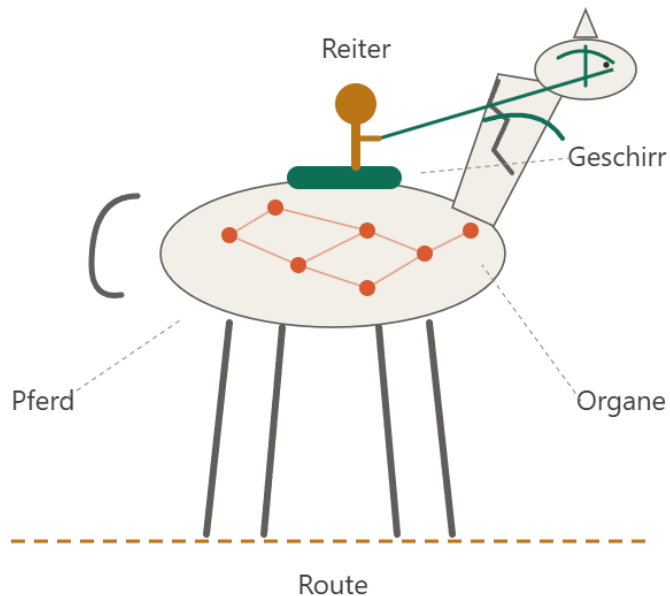
Senior-Developer



gollnickdata.de

Agents

Coding Agents: Architektur



Harness

Tools · Loop · Skills · Hooks · MCP

LLM

Claude · GPT · Gemini · Qwen

Neuronales Netz

Transformer · Gewichte · Aktivierungen

Fundament



Agents

Karpathy's Four Principles

How to steer an agent: be concrete, be incremental, be skeptical.

The Principles

- Think before coding — write the plan in prose first
- Simplicity first — smallest change that moves the goal
- Surgical changes — no touching unrelated code
- Goal-driven — validate against the target, not the vibe

Why They Matter

- Agents fail silently when goals are vague
- Scope creep compounds across turns
- Broad edits explode review & rollback cost
- Without validation, success is indistinguishable from hallucination



Agents

Coding Agents: Arena



The Arena

Where AI tools fight for developer love

Think Arena should blow up? Same.

Like on X

Coding Agents

LLMs

Live



Developer's choice



Claude Code

235 D-Index



Most loved



Claude Code

62 / 100



Fastest rising



Windsurf

+14% vs prior 24h



Most discussed



Claude Code

786 mentions



Most controversial



Claude Code

Heat 41

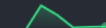
Source: <https://app.daily.dev/agents/arena>

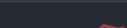
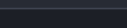
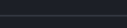
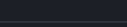


gollnickdata.de

Agents

Coding Agents: Arena

	TOOL	D-INDEX ⓘ	24H VOL ⓘ	SENTIMENT ⓘ	MOMENTUM ⓘ	7D TREND
	  Claude Code	235	786	62	+5%	
	  Codex	182	610	62	+9%	
	  Cursor	141	608	56	-18%	
4	 Antigravity	76	341	53	+10%	
5	 OpenCode	74	256	58	-34%	
6	 Copilot	33	125	54	+6%	
7	 Windsurf	23	62	56	+14%	

	TOOL	D-INDEX ⓘ	24H VOL ⓘ	SENTIMENT ⓘ	MOMENTUM ⓘ	7D TREND
	  Gemini	145	699	53	+10%	
	  Claude Opus	133	432	61	+20%	
	  Qwen	121	325	63	-16%	
4	 GPT-5	111	543	53	-2%	
5	 DeepSeek	93	446	53	+4%	
6	 GPT Codex	89	267	62	-10%	

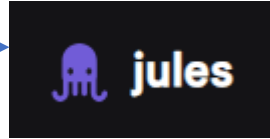
Source: <https://app.daily.dev/agents/arena>

Agents

Coding Agents: Exercise

User Prompt

Diese Anwendung liest einen kurzen Text ein und bewertet dessen emotionale Stimmung (z. B. positiv, negativ, neutral) mithilfe von Sentiment-Analyse. Basierend auf dem ermittelten Vibe generiert die App eine Webseite. Beispielsweise könnte ein positiver Text eine helle, freundliche Webseite mit vielen leuchtenden Farben und optimistischen Sprüchen erzeugen, während ein negativer Text eine düstere, minimalistische Seite mit gedämpften Farben und melancholischen Zitaten generiert. Der Code wird in Python geschrieben und streamlit für die App verwendet.



- 1 Erstellen der Projektdateien: Ich erstelle die Hauptanwendungsdatei `app.py` und die Abhängigkeitsdatei `requirements.txt`.
- 2 Abhängigkeiten installieren: Ich installiere die in `requirements.txt` aufgeführten Bibliotheken (`streamlit` und `textblob`).
- 3 Implementieren der Kernlogik: Ich schreibe den Python-Code in `app.py`. Dies beinhaltet die Einrichtung der Streamlit-Benutzeroberfläche, die Integration der Sentiment-Analyse mit `TextBlob` und die Logik zur dynamischen Generierung von Webseiten basierend auf der erkannten Stimmung.
- 4 Testen der Anwendung: Ich führe die Streamlit-Anwendung aus, um sicherzustellen, dass sie wie erwartet funktioniert. Ich werde Texte mit unterschiedlichen Stimmungen eingeben, um die generierten Webseiten zu überprüfen.
- 5 Code einreichen: Sobald die Anwendung voll funktionsfähig ist, reiche ich den Code ein.

Approve plan? →



Agents

Coding Agents: Exercise



```
requirements.txt
app.py

/app.py
1
2 + import streamlit as st
3 + from textblob import TextBlob
4 +
5 + def generate_website(sentiment):
6 +     """Generiert eine Webseite basierend auf der Stimmung."""
7 +     if sentiment == "positiv":
8 +         st.markdown("""
9 +             <style>
10 +             .container {
11 +                 background-color: #ffecb3;
12 +                 color: #333;
13 +                 padding: 20px;
14 +                 border-radius: 10px;
15 +                 text-align: center;
16 +                 font-family: 'Arial', sans-serif;
17 +             }
18 +             h1 {
19 +                 color: #ff8f00;
20 +             }
21 +         </style>
22 +         <div class="container">
23 +             <h1>Ein sonniger Tag! ☀️</h1>
24 +             <p>Die Stimmung ist wunderbar positiv. Hier ist ein Lächeln für dich! 😊</p>
25 +             <p><i>"Behalte dein Gesicht immer im Sonnenschein - und Schatten werden hinter dich"</i>
26 +         </div>
27 +         """ , unsafe_allow_html=True)
28 +     elif sentiment == "negativ":
29 +         st.markdown("""
30 +             <style>
31 +             .container {
32 +                 background-color: #424242;
33 +                 color: #f5f5f5;
34 +                 padding: 20px;
35 +                 border-radius: 10px;
36 +                 text-align: center;
37 +                 font-family: 'Georgia', serif;
38 +             }
39 +             h1 {
40 +                 color: #9e9e9e;
41 +             }
42 +         </style>
43 +         <div class="container">
44 +             <h1>Ein trüblicher Tag! ☁️</h1>
45 +             <p>Die Stimmung ist leider negativ. Hier ist ein Trauern für dich! 😞</p>
46 +             <p><i>"Behalte dein Gesicht immer im Regen - und Schatten werden hinter dich"</i>
47 +         </div>
48 +         """ , unsafe_allow_html=True)
```

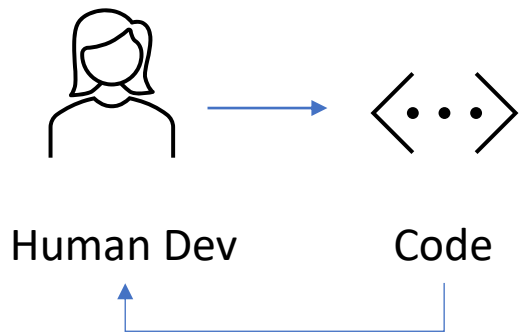
```
requirements.txt
app.py

/app.py
1
2 + streamlit
3 + textblob
```

Agents

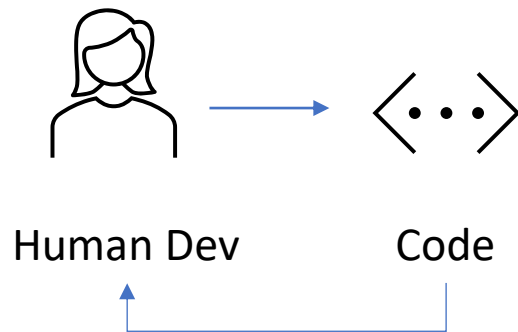
Coding Agents: World before Agents

Feature 1



...

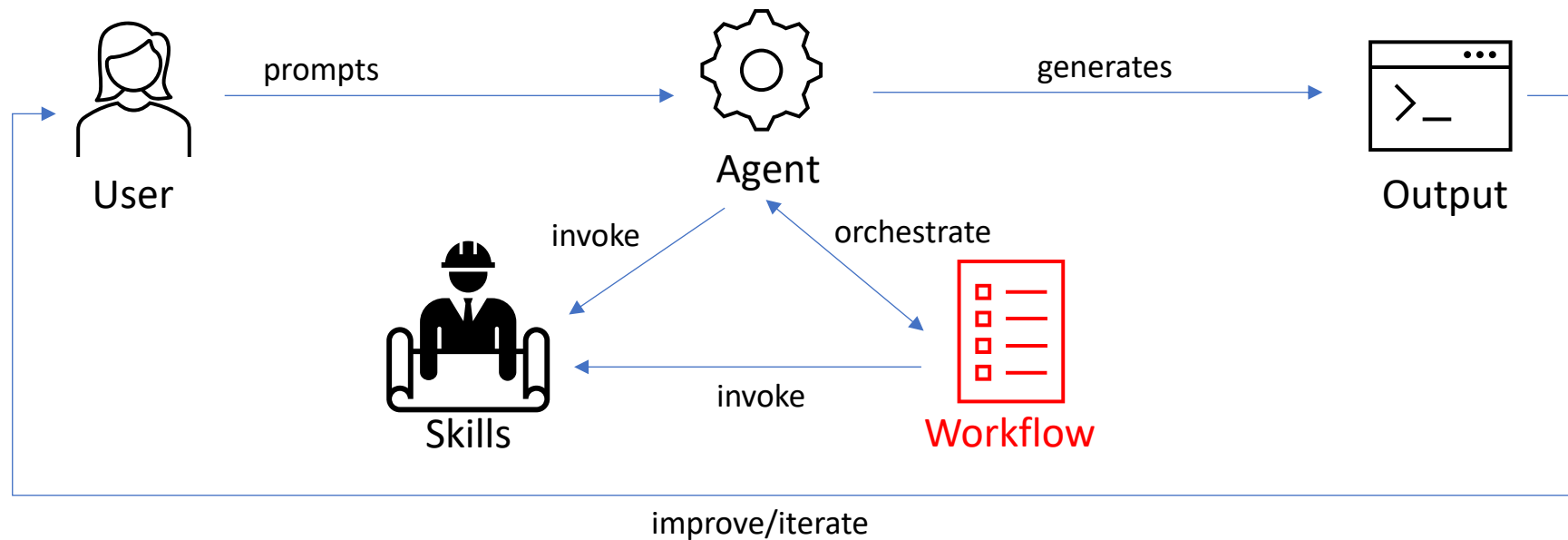
Feature N



Agents

Agentic Coding: Workflow

Agentic Coding



The difference between vibe and agentic coding is discipline.



gollnickdata.de

Agents

Agentic Coding: Workflow

Why?



Apollo guidance software

Source: <https://news.mit.edu/2016/scene-at-mit-margaret-hamilton-apollo-code-0817>

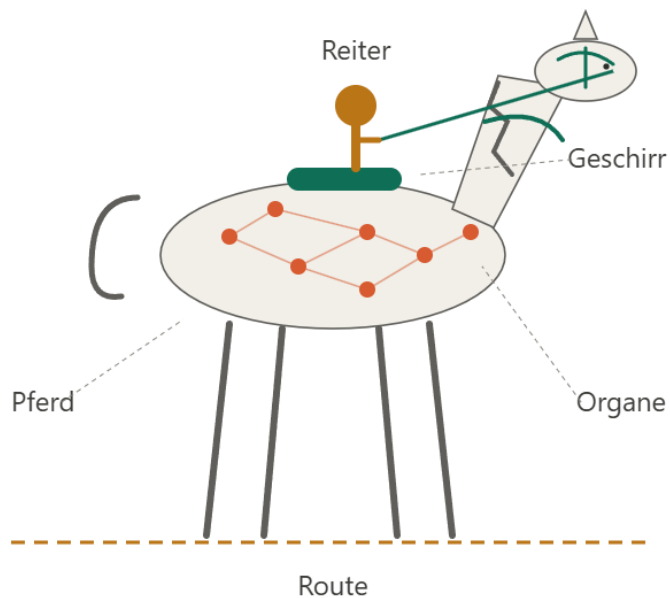
- Apollo program did have any error margin
- More critical environments require stricter guidelines and processes
- NASA developed rules to follow
- Software devs use gateways and pipelines



gollnickdata.de

Agents

Coding Agents: Architektur



Workflow

BMAD · SpecKit · GSD · Superpowers

Harness

Tools · Loop · Skills · Hooks · MCP

LLM

Claude · GPT · Gemini · Qwen

Neuronales Netz

Transformer · Gewichte · Aktivierungen

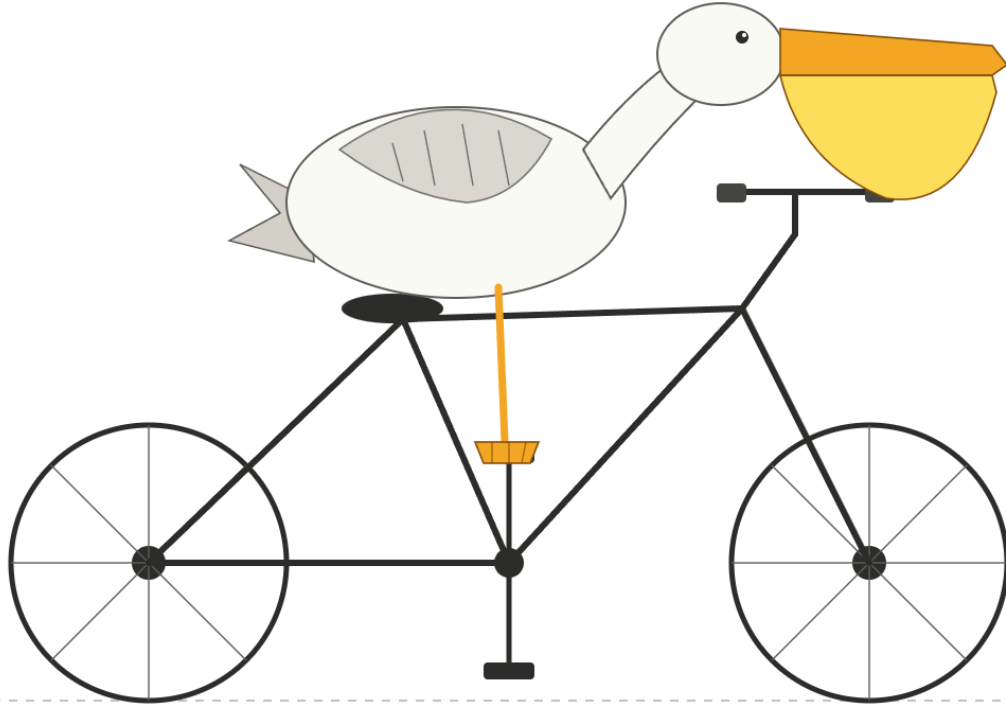
Fundament



gollnickdata.de

Agents

Coding Agents: Architektur



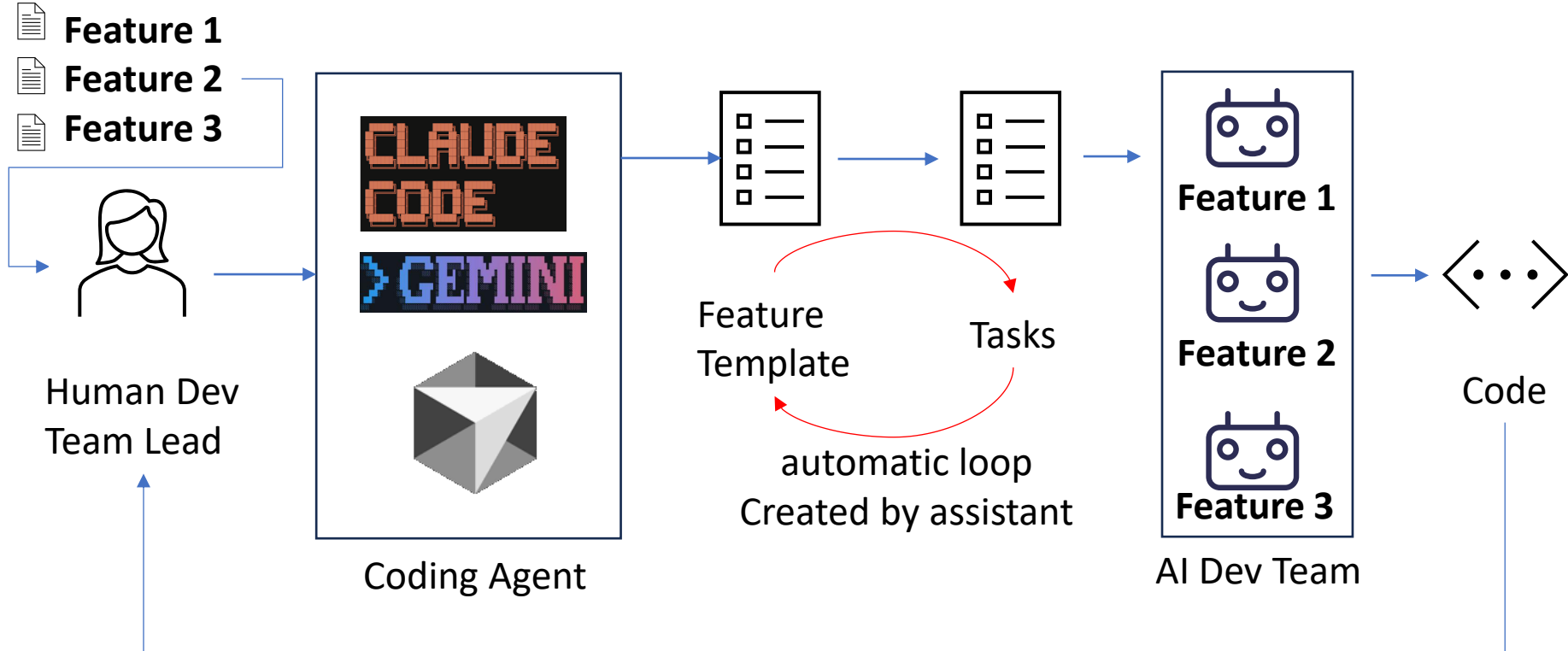
Pelican on a bicycle — Claude Opus 4.7



gollnickdata.de

Agents

Agentic Coding: Workflow



User starts new loop, when needed

Agents

Agentic Coding: Workflow

BMAD

- Breakthrough Method for Agile AI-Driven Development
- 12+ Personas
- Simulates agile development team
- 4-phase agile cycle with Analysis, Planning, Solution, Implementation
- 30+ workflows

SpecKit

- CLI commands
- „specifications don't serve code, code serves specification
- Gated process: constitution, spec, plan, tasks, implement

GSD

- Get Shit Done
- Specialised subagents: researcher, planner, executor, verifier
- Creates artifacts like spec.md, plan.md, research.md, ...

Superpowers

- Forces structured dialogue between developer and AI agent
- Gated pipeline: brainstorm, TDD, implement, review
- Quality enforcement



Agents

Validation Loop as First-Class Concern

Meta Staff Engineer insight: build/test/lint isn't an afterthought — it's the product.

The Loop

- 1. Agent proposes a change
- 2. Validation runs (build, test, lint, typecheck)
- 3. Failures feed back as concrete errors
- 4. Agent iterates until green — or asks for help

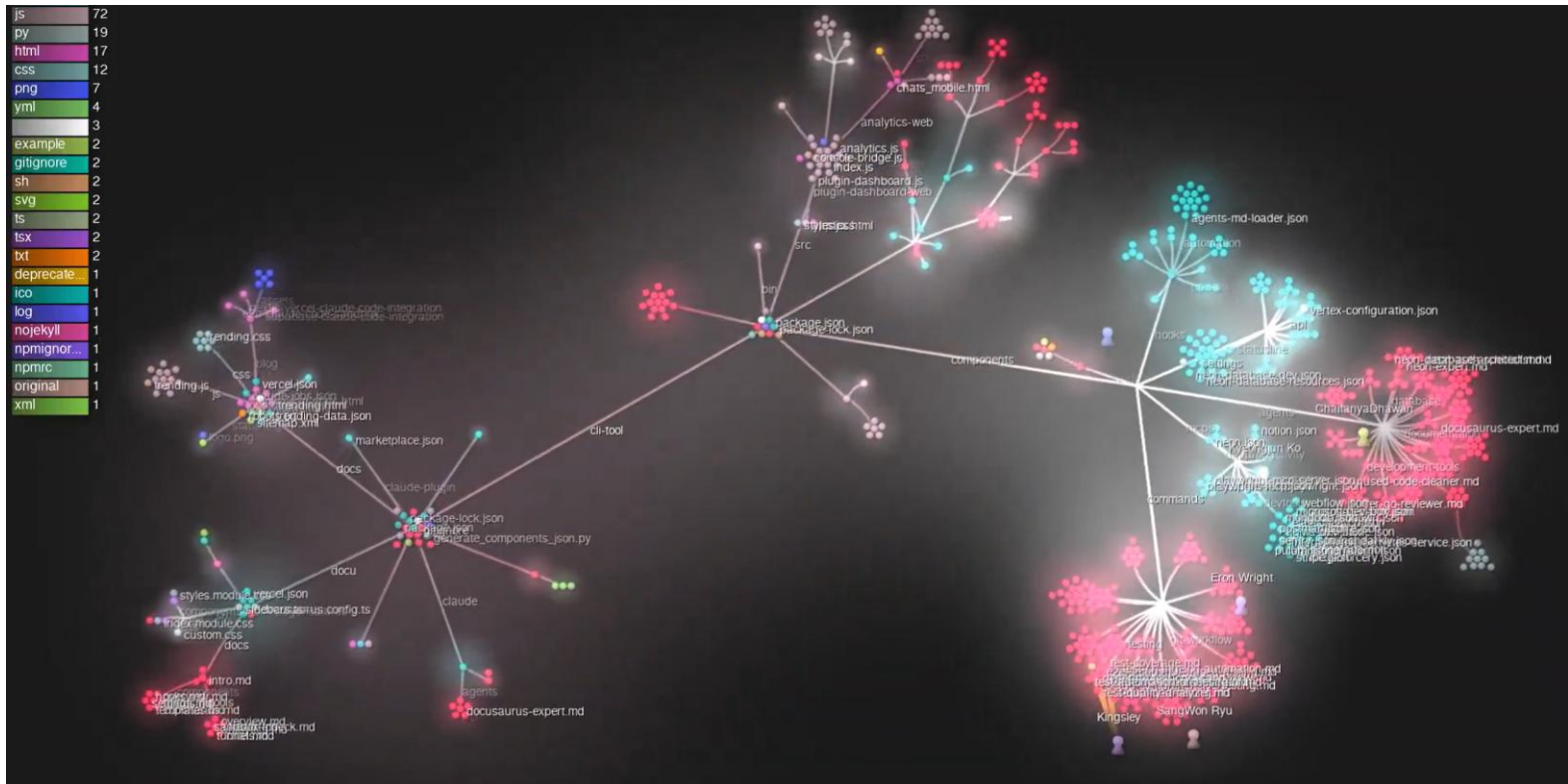
Design Implications

- Fast feedback beats smart prompts — optimize for loop speed
- Red-green signals must be unambiguous
- If validation is slow or flaky, the agent gets lost
- Invest in your test suite before investing in your prompts



Agents

Coding Agents: Claude Code Visualisation



Source: https://x.com/dani_avila7/status/1978510902740877344

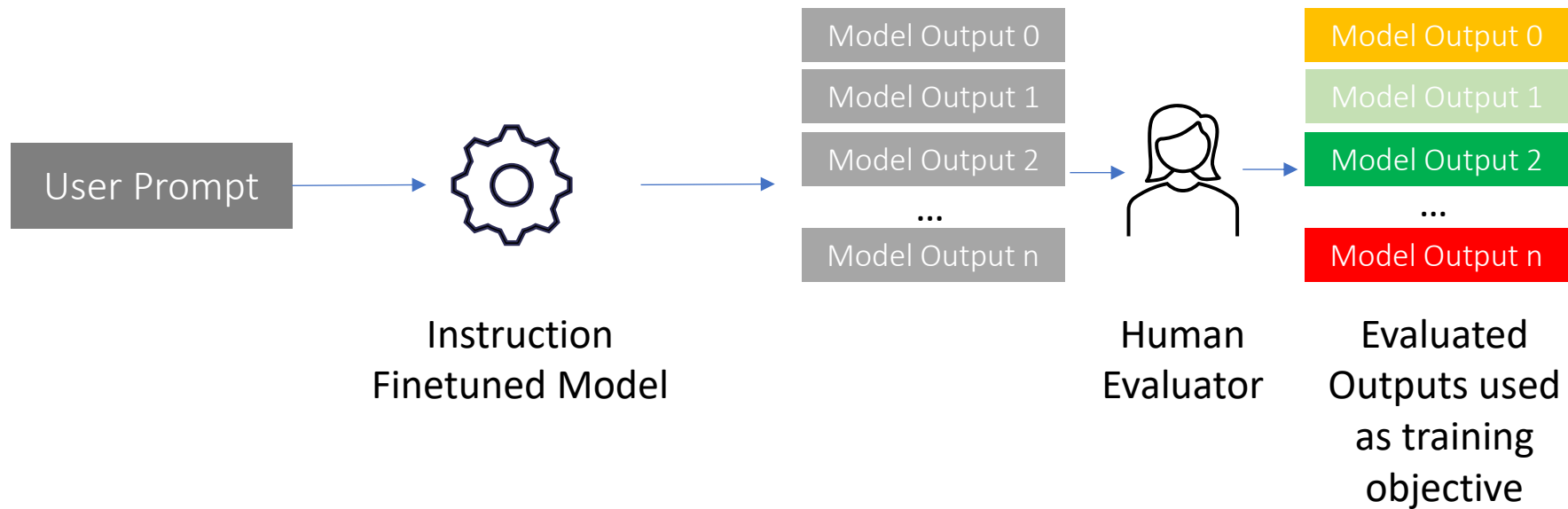


gollnickdata.de

How do Coding Agents work?

Large Language Models

Training Process: Reinforcement Learning from Human Feedback



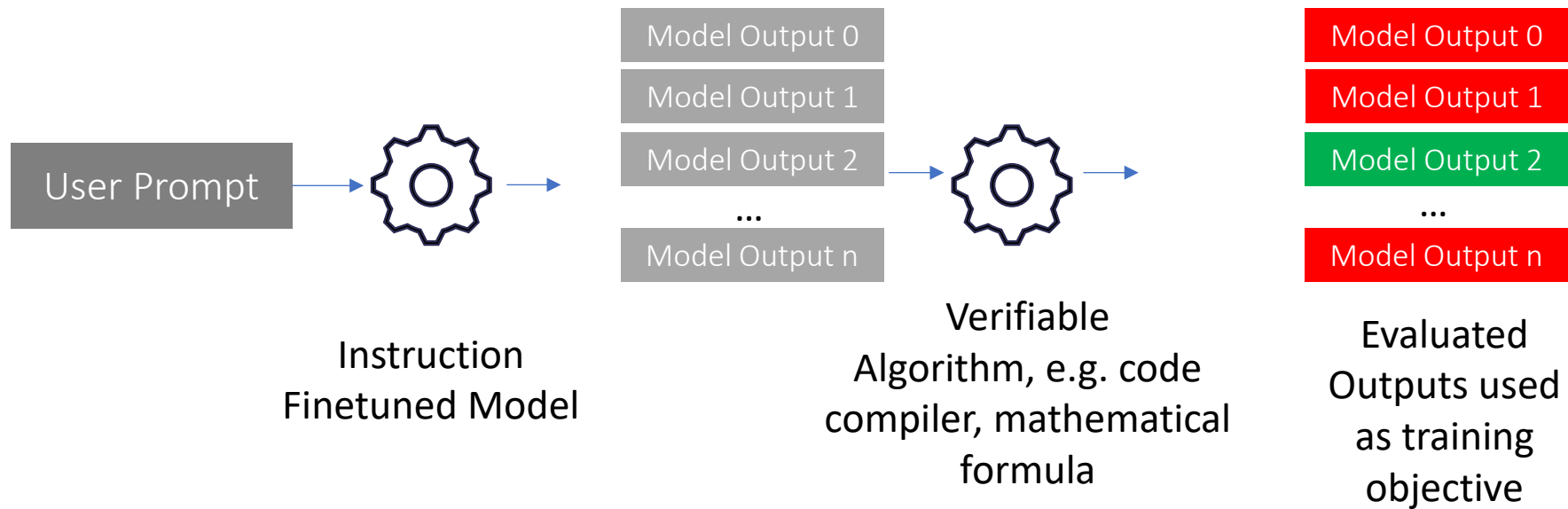
The model predicts what a human would answer.



gollnickdata.de

Large Language Models

Training Process: Reinforcement Learning from Verifiable Rewards (RLVR)



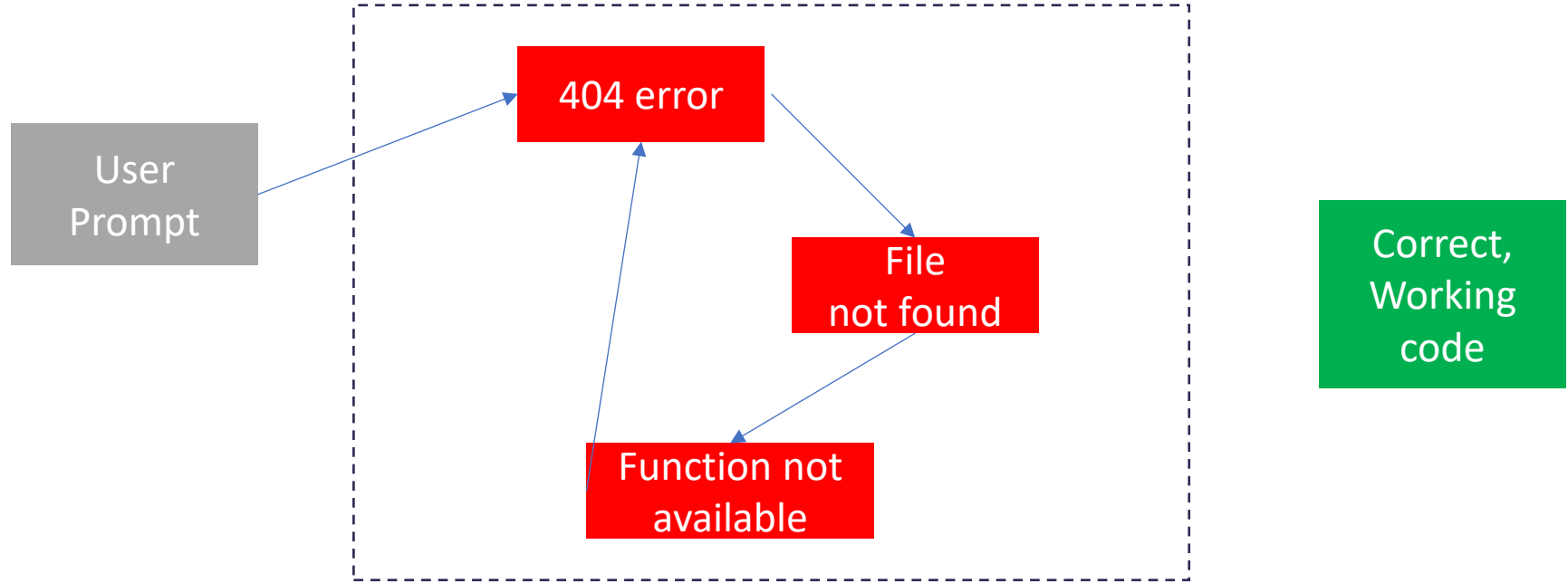
AI only gets rewarded for 100% correct answers



gollnickdata.de

Agents

Issue: Semantic Drift

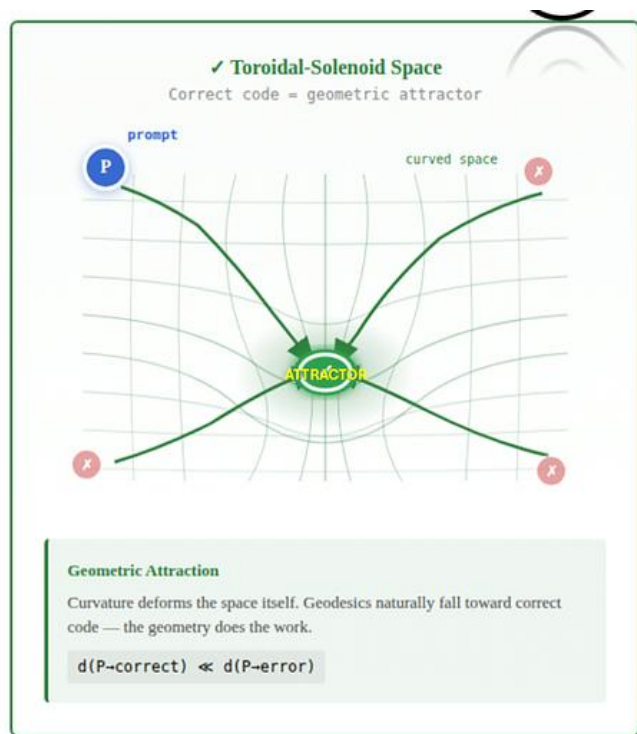


Errors are scattered and not clustered. Agent cannot differentiate poor from good solutions.



Agents

Solution: Curved Manifolds



When Models Manipulate Manifolds: The Geometry of a Counting Task

Wes Gurnee*, Emmanuel Ameisen*, Isaac Kauvar, Julius Tarng,

Adam Pearce, Chris Olah, Joshua Batson*†
Anthropic

*Equal contribution. †Corresponding author: joshb@anthropic.com

<https://arxiv.org/html/2601.04480v1>

- Errors are uphill
 - Correct solutions are uphill
- Coding agent is attracted by correct solutions

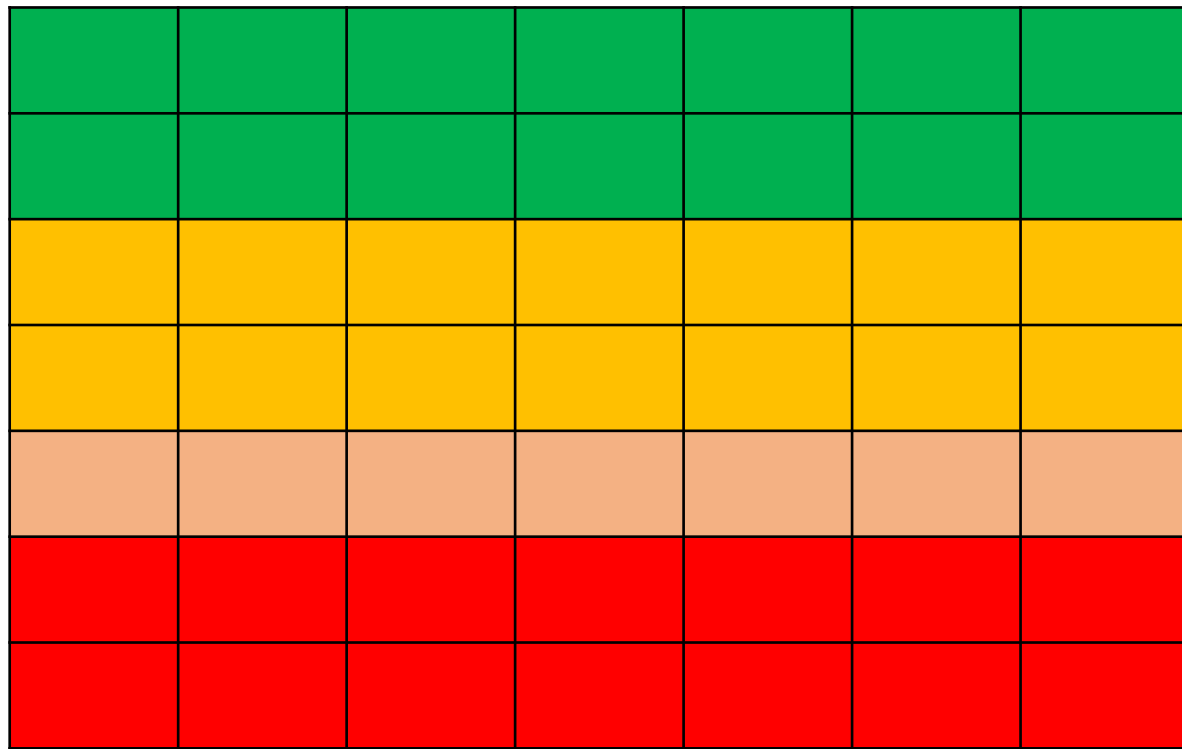
Sources: <https://ai.gopubby.com/you-all-are-wrong-using-ai-for-programming-until-now-42a00e6a8e88>



gollnickdata.de

Agents

Context Rot: The Needle in the Haystack Problem



0-30%: peak quality

>30%: model skips details,
makes shortcuts

>70%: hallucinations increase

>80%: earlier discussed
requirements are forgotten

Context Window



gollnickdata.de

Agents

Distractors: The Silent Failure Mode

Similar-but-wrong information pulled confidently into context poisons the entire run.

What Happens

- Retrieval surfaces near-matches that look authoritative
- Agent treats them as ground truth
- Downstream reasoning compounds the error
- Symptoms look like a model failure, not a context failure

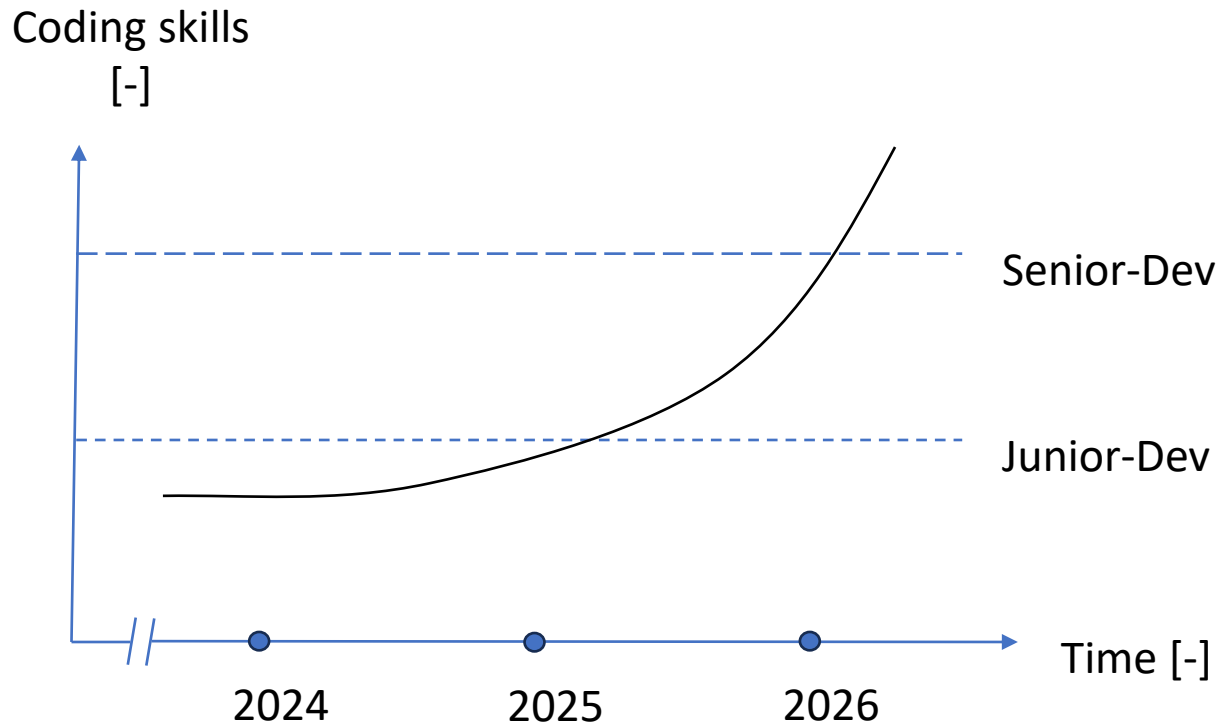
Common Sources

- Outdated docs next to current docs
- Similar function names across modules (fetchUser vs getUser)
- Legacy APIs shadowing the new ones
- Deprecated patterns in Stack Overflow answers



Agents

Coding Agents: Capabilities



Source: own graphic



gollnickdata.de

Claude Code: Introduction

Claude Code

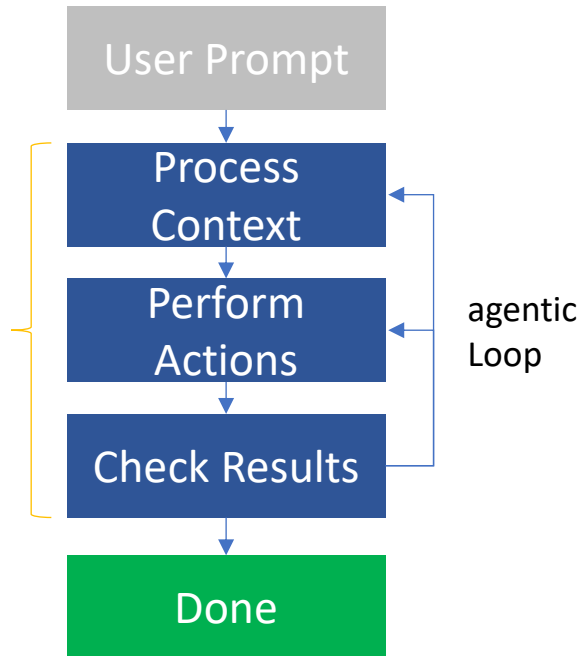
What is it?

An AI-Coding-Tool, that originally ran only in the terminal

Core Functions

- Develop features based on descriptions
- Debug
- Navigate in Codebase
- Manage Git-Workflows

User can interrupt,
Change progress



Claude Code can edit files directly and perform commands.



gollnickdata.de

Claude Code

Installation and Setup

Requirement

- Claude Subscription (Pro/Max/Teams/Enterprise)
- Or: API Access via Claude Console

Installation

Available for MacOS, Linux, Windows

<https://code.claude.com/docs/en/setup>

Add to path variable

Start

Run `claude` in Terminal

On first start: login required



Claude Code

Installation and Setup

Installation

- <https://code.claude.com/docs/en/quickstart>

Nutzung via API KEY

- Only relevant for participants using my API key
- Set Environment variable ANTHROPIC_API_KEY on system level
- Powershell: `function claude-low { & claude --model claude-3-5-haiku-latest $args }`
- Sets the model to cheap option

Demo



Claude Code

Change CLAUDE CODE to run with custom LLM

Since Ollama v0.14 — no proxy, no translation layer

Claude Code talks directly to localhost:11434

1 · Point Claude Code at Ollama

Set three environment variables:

```
export ANTHROPIC_BASE_URL=  
http://localhost:11434  
export ANTHROPIC_AUTH_TOKEN=ollama  
export ANTHROPIC_API_KEY=""
```

Then: `claude --model qwen2.5-coder:14b`

2 · Fix the KV-cache gotcha

Attribution header invalidates cache

→ ~90 % slower without this fix

Edit `~/.claude.json`:

```
{  
  "CLAUDE_CODE_ATTRIBUTION_HEADER": 0  
}
```

Recommended models

qwen2.5-coder:7b — 16 GB RAM
qwen2.5-coder:14b — sweet spot, 24 GB
qwen3-coder:30b — agentic, 48 GB RAM
devstral-small:24b — RTX 4090

Minimum: 32K context · 64K+ recommended

When this setup shines

DSGVO / KRITIS — code stays local
Air-gapped dev on customer premises
High-volume tasks without API costs
Experimentation with open models

Switch back: `unset ANTHROPIC_BASE_URL`



Claude Code

Installation and Setup

Create PRD.md

- Product requirement document

Start Claude

- claude

Implement
first version

- /init
- Implement @ai_docs\PRD.md

Demo



Claude Code

Permission Modes

Demo

plan

„ultrathink“ für besonders komplexe Aufgaben

Read-only

Claude erkundet,
liest, plant —
aber schreibt nichts

Für Architektur-
Diskussionen

default

Fragt jedes Mal

Schreib- und Bash-
Operationen brauchen
Bestätigung

Für sensible
Codebases

acceptEdits

Edits ohne Frage

File-Schreiben + ein
paar Bash-Befehle
(mkdir, mv, cp ...)

Für aktives
Coding

auto

Classifier-überwacht

Separates Modell
prüft Aktionen
vor Ausführung

Team/Enterprise
Research Preview

⚠ bypassPermissions

--dangerously-skip-permissions · keinerlei Prüfung · nur in Sandbox/Container/CI

Sicherere Alternative:
in settings.json erlaubte und
verbotene Methoden anpassen

← mehr Sicherheit

mehr Geschwindigkeit →

In jedem Modus gilt: Writes auf geschützte Pfade werden nie automatisch genehmigt



gollnickdata.de

Claude Code

Workflow Process

Workflow-Process

Start in Planning mode
(SHIFT + TAB + TAB)

> Think through how to add OAuth authentication to our app. Show the architecture, files we'll need, and implementation steps. Feel free to ask me questions.

Test driven development

> Write comprehensive tests for the OAuth flow - sign up, login, token refresh, and error cases. Make sure they fail for now. Then, implement the OAuth system to make these tests pass.

Start in Edit or Auto-Mode
(SHIFT + TAB)



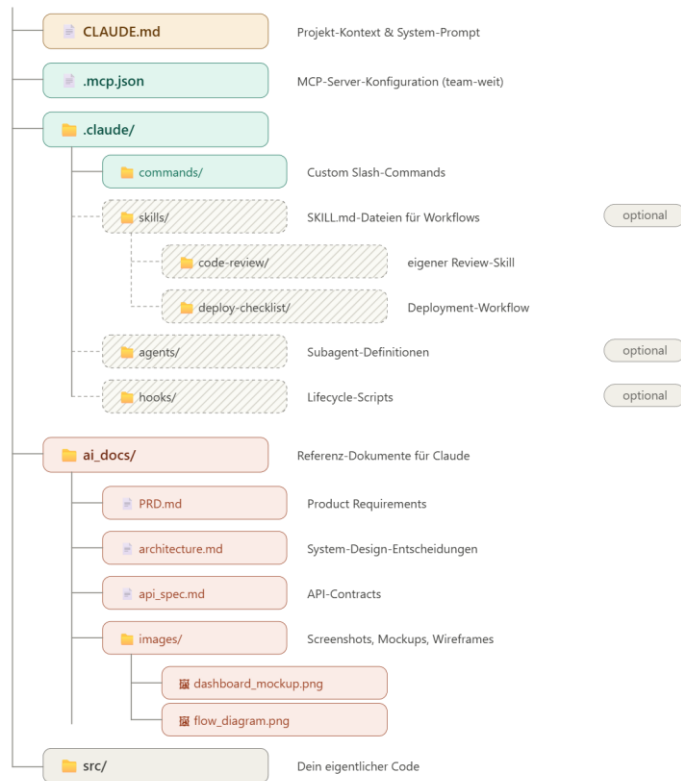
Claude Code

Installation and Setup

Folder structure

Empfohlene Projektstruktur

my-project/



Demo



gollnickdata.de

Claude Code

Claude.md

A markdown-file in the project-root folder provides important context for Claude:

Typically contains:

- Build Commands
- Code Style Guidelines
- Architecture overview
- Testing strategies

Beispiel

```
# Build Commands  
npm run build  
  
# Code Style  
Use TypeScript
```

Tipp: Check CLAUDE.md into Git!



Claude Code

CLAUDE.md Sizing — The 300-Line Rule

Meta Staff Engineer guidance: treat CLAUDE.md as a living system prompt, not a knowledge dump.

Target: ~300 lines

- Beyond that, context rot kicks in hard
- Claude starts ignoring or misreading instructions
- Signal-to-noise degrades — more text, worse results
- Every line competes for attention with the actual task

What to Cut

- Move reference docs to /ai_docs/ and link them
- Push tribal knowledge into skills or slash commands
- Delete anything that hasn't changed behavior in 3+ sessions
- If it's not altering outputs, it's taxing them



Claude Code

Slash-Commands

Shortcuts for daily tasks

Session Management

- **/clear** – Reset context
- **/compact** – reduce context
- **/resume** – Continue session

Information

- **/help** – all commands
- **/stats** – usage statistics

Utilities

- **/agents** – configure sub-Agents/Agent-teams
- **/export** – export conversation

Tipp: **/help** zeigt alle verfügbaren Commands!



Claude Code

Current Claude Code cool features

Major quality-of-life upgrades shipped in late 2025 and early 2026.

Developer UX

- Native in VS Code extension
- Drag & drop images and pdf-files for context
- # at start of line - quick-append to CLAUDE.md
- @ - reference a file path with autocomplete
- Ctrl + B to put current task into background

Under the Hood

- Skills & Plugins marketplace — installable capabilities
- Sandboxing — filesystem + network isolation by default
- Hooks API — deterministic control over lifecycle events



Claude Code

Slash Commands

Repeatable tasks on keystroke

Scripts that fire when user enter `/[command-name]`.

They are typically `.md` files `.claude/commands/`

- `.claude/commands/` — projektlokal, upload to git
- `~/.claude/commands/` — global, only for user
- In case of name conflict wins project name
- Subdirectories werden zu Namespaces:
`.claude/commands/git/commit.md`
→ `/git:commit`

Example `.claude/commands/review.md`

description: Reviewt eine Datei auf Bugs und Style-Probleme

argument-hint: [pfad/zur/datei]

allowed-tools: Read, Grep

Reviewe @\$1 auf:

- Bugs und Logik-Fehler
- Style-Verstöße gegen unsere Konventionen
- Verbesserungsvorschläge

Gib das Ergebnis als kurze Liste aus.

Usage: `/review src/auth.py`

\$Argument bzw. **\$1, \$2, \$3** . Use **@** to link to file content



Claude Code

Hooks

Scripts that fire automatically at lifecycle events — deterministic control, not AI.

Available Events

- PreToolUse / PostToolUse
- SessionStart / SessionEnd
- PromptSubmit / Compaction
- SubagentStart / SubagentStop

Use Cases

- Auto-lint after every file edit
- Run test suite after code changes
- Block dangerous commands (e.g., `rm -rf`)
- Send notifications when tasks complete



Claude Code

MCP Integration

Model Context Protocol (MCP) — an open protocol that connects Claude to external services and tools.

How it works

- Servers expose tools and resources
- Claude Code acts as MCP Client
- Configured via `.mcp.json`

Example Servers

- GitHub — PRs, issues
- PostgreSQL — DB queries
- Playwright — browser testing
- Google Workspace — Docs, Mail



Claude Code

Skills & Plugins

Skills

Reusable knowledge and workflows that Claude loads on demand — like macros for AI.

- SKILL.md files with metadata and instructions
- Auto-detected by context or invoked via /skill-name
- Reference skills (knowledge) or Action skills (workflows)

Plugins

- Bundle skills, hooks, subagents, and MCP servers into one installable unit
- Namespaced to avoid conflicts (e.g., /my-plugin:review)
- Installable via marketplace or local paths



Claude Code

Skills & Plugins

Example for a typical skill.md file

```
---
name: truck-evaluation
description: Run the full truck-detection evaluation pipeline. Use when evaluating YOLO detections against Original recording.
---
```

Evaluation Workflow

Run these phases in order. Each phase corresponds to a separate skill — invoke them by name and let Claude load each skill's SKILL.md as needed.

Phase 1: Load source data
Use the **video-loader** skill to convert .video files and extract annotations. This skill knows the proprietary format and the annotation JSON schema.

Phase 2: Sampling
Use the **stratified-sampler** skill to draw the evaluation sample across confidence bins. Important: this skill enforces the methodology in the specs— do not improvise.

Phase 3: Metrics
Use the **confusion-matrix** skill to compute precision, recall, and F1 per class, plus the failure analysis on the top-10 false positives.

Phase 4: Report
Write the markdown report under `evaluations/<recording-id>/report.md` using the template in `references/report\_template.md`.



Claude Code

Top Plugins in Practice

101+ plugins on the official marketplace as of March 2026. These are what senior devs actually install.

Most Installed

- Frontend Design — 370K+ installs, component patterns
- Superpowers — brainstorm → plan → execute workflow
- Ralph Loop — autonomous fix-test-repeat loop
- Context7 — up-to-date library docs in-context

MCP Bridges

- GitHub MCP — PRs, issues, CI from the terminal
- Playwright MCP — browser automation & screen shots
- Figma MCP — design-to-code handoff
- Security Guidance — OWASP-aware review skill



Claude Code

Checkpoints & Sandboxing

Checkpoints

- Auto-saves code state before each change
- Rewind with Esc+Esc or /rewind command
- Restore code, conversation, or both
- Enables ambitious refactors with confidence

Sandboxing

- Filesystem isolation for project files
- Network isolation via proxy server
- OS-level enforcement (bubblewrap / seatbelt)
- Reduces permission prompts by 84%



Agentic Coding Levels

Claude Code

The 7 Levels of Agentic Coding Mastery

Level 1 Executor · CLI Prompts

Level 2 Resident · Context Engineering

Level 3 Specialist · Skills & Hooks

Level 4 Connector · MCP Servers

Level 5 Supervisor · Subagents

Level 6 Manager · Agent Teams

Level 7 Architect · CI/CD Integration



Claude Code

Levels 1–2: Foundation

Level 1: Executor · CLI Prompts

The Core Foundation

- Treat Claude Code like a powerful terminal
- One-off tasks: claude "fix the bug in auth.py"
- 200k+ token context window for instant project awareness
- No copy-pasting needed — reads local files directly



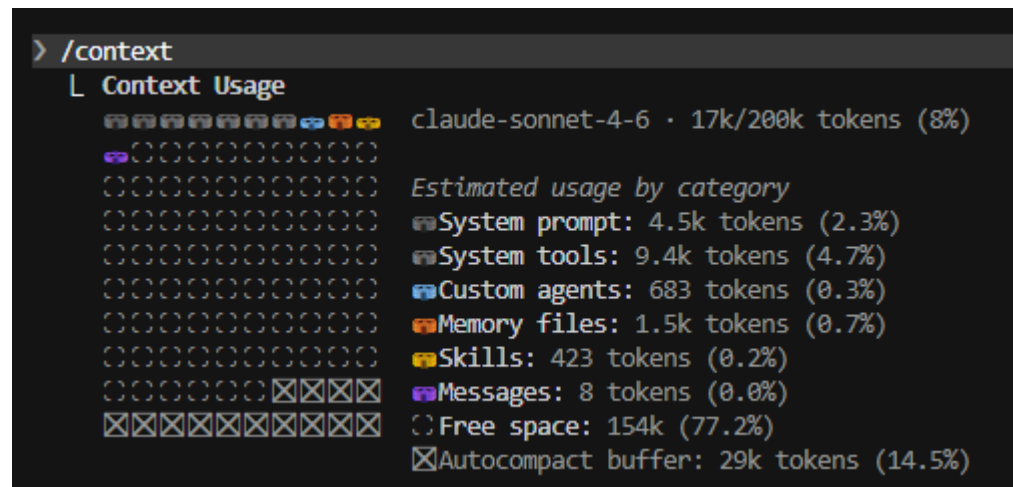
Claude Code

Level 2: Context Management

Level 2: Resident · CLAUDE.md

Personalized Memory and Context Management

- CLAUDE.md = System Prompt for your repo
- Define project rules, naming conventions, preferred libraries
- Read at every session start — no repeated instructions
- Transforms generic AI into a customized teammate
- Understand the importance of context



Output of /context



Claude Code

Level 2: Context Management

CLAUDE.md und MEMORY.md im Vergleich.

CLAUDE.md

- Autor: Du
- Zweck: deine Anweisungen
- Geladen: zu Session Beginn

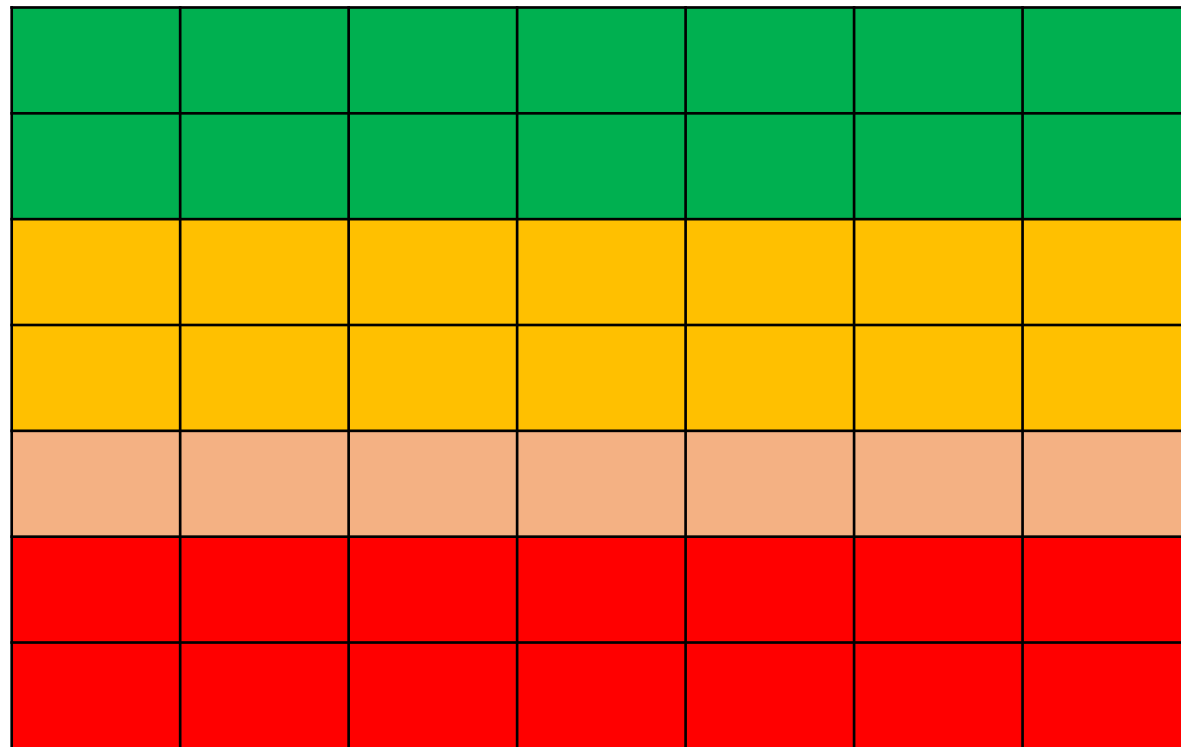
MEMORY.md

- Autor: Claude
- Zweck: eigene Notizen von Claude
- Geladen: zu Session Beginn



Claude Code

Level 2: Context Management – Context Rot



0-30%: peak quality

>30%: model skips details,
makes shortcuts

>70%: hallucinations increase

>80%: earlier discussed
requirements are forgotten

Context Window

Claude Code

Levels 3–4: Automation & Integration

Level 3: Specialist · Skills & Hooks

Repeatability

- Skills: Markdown guides Claude invokes for complex workflows
- Hooks: Auto-trigger shell commands (e.g., npm test after edits)
- Stop explaining how — just tell Claude what to do
- Custom slash commands like /simplify or /debug

Level 4: Connector · MCP Servers

External Integration

- MCP breaks Claude out of the local terminal
- Fetch data from Jira, Slack, Google Docs, databases
- AI becomes environment-aware, not just code-aware
- Connect to your full ecosystem via MCP-protocol



Claude Code

Levels 5–6: Multi-Agent Orchestration

Level 5: Supervisor · Subagents

Parallel Delegation

- Spawn multiple Claude subagents
- Each handles an independent task
- Parent coordinates & merges results
- Great for refactors & test suites

Level 6: Manager · Agent Teams

Structured Team Workflows

- Define roles: planner, coder, tester
- Agents hand off work in a pipeline
- Mimics real team dynamics with AI
- Scale beyond single-agent capacity



Claude Code

Git Worktrees — Parallelism That Actually Scales

Running Claude on main and a feature branch at the same time? Worktrees are the answer.

The Setup

- `git worktree add ../feat-x feat-x`
- Each worktree → isolated filesystem + ports + deps
- Spawn one Claude instance per worktree
- No branch-switching, no stashing, no contention

Why It's 10x, Not 2x

- Agents don't context-switch the way humans do
- Three agents on three worktrees = three full-speed streams
- Merge conflicts surface at PR time, not mid-flow
- The constraint becomes your review capacity



Claude Code

Cost Architecture — The Advisor Pattern

Pair a cheap executor with an expensive advisor. Both see the same context, only one does the bulk work

The Pattern

- Executor — Haiku or Sonnet, handles most turns
- Advisor — Opus, called for hard calls only
- Shared CLAUDE.md so they reason about the same project
- Executor escalates; advisor never executes directly

When It Pays

- Long sessions where 80% of turns are mechanical
- Complex refactors with occasional architectural decisions
- Teams doing hundreds of runs per day
- Rule of thumb: 5–10x cost reduction at comparable quality



Claude Code

Level 7: Full Automation

Level 7: Architect · CI/CD Integration

Autonomous Pipeline

- Claude runs headless inside CI/CD pipelines (GitHub Actions, GitLab CI)
- Triggered by events: new issues, PRs, scheduled jobs, deploys
- Zero human intervention — fully autonomous coding workflows (e.G. Ralph-Wiggum)
- Auto-generate PRs, run tests, fix failures, update documentation
- The ultimate goal: AI as a self-sustaining engineering partner



Claude Code

Headless Claude Code – for automation

-p / --print flag

Turns the interactive CLI into a subroutine

Use cases: bash scripts, pre-commit hooks, GitHub Actions, linters, reviewers

Three output formats

text

Default · prose

Human-readable,
unusable for scripts

```
claude -p "Explain  
this function"
```

json

Single envelope at the end

With result, session_id,
total_cost_usd, duration

```
claude -p "Review"  
--output-format json
```

stream-json

NDJSON · real-time

One event per line
→ live dashboards, audit

```
--output-format  
stream-json --verbose
```

Common patterns

```
git diff main | claude -p "report typos as filename:line" --output-format json
```

Session continuation: claude --continue or --resume with session_id from JSON

--allowedTools "Read,Edit" for file operations without permission prompts

--append-system-prompt "You are a security reviewer" for role specialization



Forecast

Forecast

Remote Control via Phone

Both paths solve the same problem differently

Remote Control routes through Anthropic · Tailscale stays inside your private network

A · Remote Control

Native Anthropic feature

Setup

1. Update to v2.1.51 or later
2. Install Claude app on phone
3. Sign in with claude.ai account
4. Run `claude remote-control`
5. Scan QR code shown in terminal

Three invocation modes

`claude remote-control` (server)
`claude --remote-control` (interactive)
`/remote-control` (from inside session)

Bonus

Push notifications via `/config`
"notify me when tests finish"

Constraints

Requires claude.ai subscription

B · Tailscale + tmux

DIY path · stays out of Anthropic infra

Setup

1. Install Tailscale on both devices
2. Enable SSH on laptop
3. `tmux new -s claude` → run claude
4. Install phone client (Termius/Termux)
5. `mosh laptop`, then `tmux attach`

Building blocks

Tailscale (private WireGuard mesh)
SSH (laptop must run an SSH server)
tmux (keeps session alive on detach)
mosh (survives WiFi switches)

Bonus

Works with API keys, not just claude.ai
No outage limit, no broker dependency

Constraints

Pick A when

You have a claude.ai subscription · You want zero infrastructure setup
You're fine with traffic going through Anthropic · Five-minute time budget

Pick B when

DSGVO / KRITIS / on-prem requirements forbid third-party brokering
You use API keys instead of a claude.ai login · You want true private networking
You need sessions that survive longer than 10-minute network outages



gollnickdata.de

Forecast

Token Economics — Why Context Is Expensive

The compounding cost most teams don't measure until the bill arrives.

The Re-Read Problem

- Every turn re-reads the full chat history
- 100-message session → ~98.5% of tokens are re-reads
- Cost grows quadratically with session length
- Long sessions burn budget before producing output

Mitigations

- Compaction — summarize old turns, keep recent verbatim
- Subagents — fresh context for independent subtasks
- Skills — load capabilities on demand, not upfront
- Checkpoints — rewind instead of re-explaining



Forecast

WISC — Four Levers for Context

Write & Isolate

- Write — persist state outside the context window
- Isolate — give each subagent its own clean window
- Use scratchpads, `/ai_docs/`, and subagent spawning
- Protect the main thread from noise

Select & Compress

- Select — pull only what this turn actually needs
- Compress — summarize aggressively, link to detail
- Progressive disclosure beats front-loaded dumps
- Skills and MCP are Select mechanisms



Forecast

Karpathy's LLM vs OB1 — persistent Knowledge Base

Karpathy's compiler analogy

Notes are source code,
the wiki is the compilation

Pipeline:

Raw notes → Wiki pages → Index → Query

No RAG, no vector store
Just clean Markdown structure
Human-readable = LLM-readable

Nate B. Jones OB1 — implementation

Open-source pattern,
deployable in under an hour

Stack:

Supabase + pgvector + MCP server

Four tools: semantic_search,
browse, stats, capture
Multi-client (CC, Cursor, ChatGPT)

Concrete variant: Obsidian vault as a starting point

One vault with three subfolders: raw / wiki / outputs

Raw = dumping ground for research and chats with Claude Code

Wiki = reference articles compiled by Claude Code from the raw layer

Outputs = slides, reports, posts based on the wiki articles

Workshop heuristic

Start with an Obsidian vault, upgrade to an MCP server when scale demands it



gollnickdata.de

Forecast

„Agentic OS“

The term is mostly marketing

It bundles existing primitives (Skills, Slash Commands, Hooks, headless mode) under a new name

Real advantages

Reproducibility — same input,
comparable output every time

Tacit knowledge becomes codified
and survives beyond your attention

Onboarding for non-coders via
clickable headless-mode buttons

Cost and time become measurable
— you can prove the ROI

Cognitive offloading — capacity
freed up for the actual work

Limitations and caveats

Dashboard claim is oversold —
terminal is faster for most solos

"Track and optimize" rarely
happens — structure gives 80%

LLMs are non-deterministic —
structural, not byte-level repro

One-click triggers without
permission checks are risky

Setup overhead only pays back
at scale — solo with one task
per week is overkill

Build it when at least two apply

You repeat the same task weekly · You share workflows with teammates or clients

You bill clients · You need audit trails · You're building for years, not weeks

Otherwise: a few CLAUDE.md files and a handful of slash commands are enough



gollnickdata.de

Socio-economic Impact

Socio-Economic Impact

Jobs

AI & The Future of Work: Which Jobs Will Change the Most?



Artificial intelligence is reshaping the global workforce. The greatest impact is on roles involving repetitive, rule-based tasks, while jobs requiring physical dexterity, complex problem-solving, and deep human empathy remain more secure. The key trend is not just job replacement, but a fundamental evolution of job roles, where AI acts as a tool or assistant.

High-Impact Zone: Jobs Being Transformed by AI

Customer Service & Telemarketing

AI chatbots and virtual assistants can now handle routine customer queries 24/7.



Content & Language Professionals

Generative AI's ability to create and translate text is transforming roles like writers and translators.



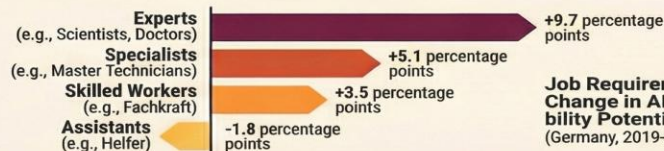
Junior Software Developers

AI coding assistants are automating basic code, shifting the entry point for tech careers.



95%
of Data Entry Clerk Jobs at Risk

AI excels at processing structured data, heavily impacting repetitive clerical and bookkeeping roles.



Job Requirement Level & Change in AI Substitutability Potential
(Germany, 2019-2022)

Low-Impact Zone: Jobs Remaining Human-Centric



Skilled Trades & Manual Labor

Jobs requiring physical dexterity and on-site problem solving (e.g., roofers, electricians) are difficult to automate.



Healthcare & Personal Care

Roles centered on empathy, direct patient care, and social work remain securely human.



Strategic & Creative Leadership

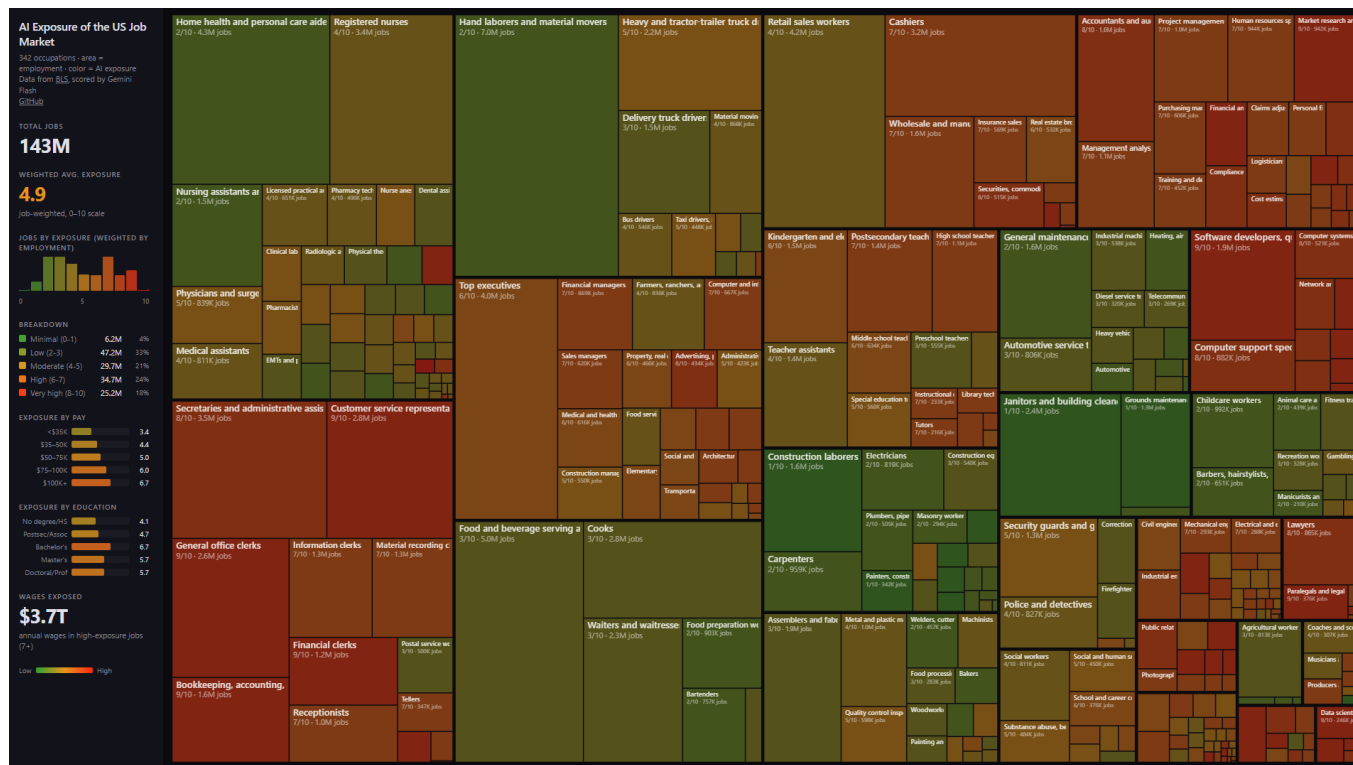
While AI is an ideation tool, complex strategy and true innovation still require human insight.

Education & Childcare

Teaching and caring for others relies on emotional intelligence and interaction that AI cannot replicate.



Jobs – AI Exposure



Source: Andrzej Karpathy, <https://joshkale.github.io/jobs/>

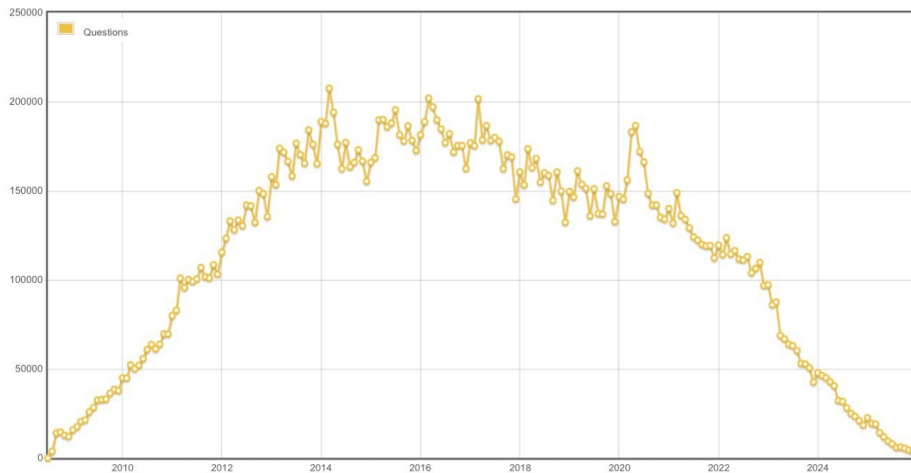


gollnickdata.de

Socio-Economic Impact

Jobs – Impact on Software Development

Stackoverflow Use



Adobe Stockprice

30.03.2026 - Kurs: 210,05



Sources: https://www.reddit.com/r/singularity/comments/1q4llh1/stackoverflow_graph_of_questions_asked_per_month/

Comdirekt, Adobe stock price, 30.03.2026

Summary

Claude Code

Best Practices

Context Management

Nutze **/clear** häufig zwischen Tasks

Mehrstufiger Workflow

Research → Plan → Implement → Test → PR

CLAUDE.md pflegen

Dokumentiere Commands, Style, Testing

Custom Commands sparsam

Nur für häufige Workflows - Natural Language bevorzugen

Sub-Agents für große Tasks

Spezialisierte Agents für komplexe Multi-Step-Ops



Claude Code

Summary

Claude Code ist...

- Agentisches Coding-Tool
- Direkt im Terminal
- Mit Projekt-Context
- Erweiterbar durch MCP
- Anpassbar durch Commands

Key Features

- Natural Language Interface
- Slash Commands
- CLAUDE.md für Context
- Sub-Agents
- Skills für Automatisierung

Starte mit **claude** im Terminal!

Dokumentation: code.claude.com/docs

